SEN381 — SOFTWARE ENGINEERING 381

CIVICCONNECT PROJECT — MILESTONE 3

Controlled Construction, Integration, Quality & Release Readiness

Raw mark: 50 | Team evidence: 30 | Individual engineering defence: 20

Milestone purpose
M3 is not an enterprise-scale production certification. It is a student-capacity engineering milestone. Your team must show that the M2 design has become a controlled, working and integrated release candidate, and that you can support important quality claims with traceable evidence.

Central engineering question
Can the team demonstrate, with manageable and credible evidence, that CivicConnect has been constructed, integrated and verified well enough to become a credible release candidate — while clearly identifying what still remains uncertain or incomplete?

1. What M3 Is — and What It Is Not

M3 expects	M3 does NOT expect
A working integrated increment/release candidate covering the agreed core scope	A production-scale municipal platform
Evidence that important M2 decisions were implemented or formally changed	Perfect adherence to M2 when implementation evidence justifies a change
A small, meaningful automated test suite	Hundreds of tests simply to increase a number
Appropriate black-box/integration/system tests	Every test technique forced into the project whether applicable or not
A basic CI pipeline with useful checks	Enterprise DevOps infrastructure
A real staging/test deployment or equivalent accessible environment	Mandatory public production deployment
A small, justified performance/load exercise	Thousands of real users or expensive commercial tooling
A release-readiness judgement supported by evidence	A claim that the system is defect-free

2. Relationship to M2 and Weeks 3–5

M2 established the Architecture, Technology & Initial Design Baseline. M3 continues the same project record: the team implements and verifies that baseline while evolving the existing PED v2.0 into PED v3.0 and extending the existing RTM with implementation and verification evidence. Where implementation exposes a problem, the team may change a decision, but the change must be controlled, traceable and justified.

Learning already completed	How it appears in M3
Week 3 — design, persistence, APIs, maintainable construction	Implemented design and integration evidence
Week 4 — change/configuration management, CI, quality gates, peer review	Repository history, PR control, CI and controlled change
Week 5 SO25 — risk-based quality strategy	Prioritisation of important tests/evidence
Week 5 SO26 — white-box/static/code assurance	Automated tests, static analysis, code review, meaningful coverage evidence
Week 5 SO27 — black-box/integration/system validation	EP/BVA/decision/state techniques where appropriate; API/integration/E2E evidence
Week 5 SO28 — automation, performance and release evidence	Regression automation, basic load/performance evidence, release-quality review and residual risk

3. Minimum Viable M3 Submission

Manageability rule

If your team can produce the following minimum evidence well, you can satisfy the core intent of M3. Extra volume does not automatically earn extra marks.

Area	Manageable minimum expectation
Working product	One integrated release candidate covering the agreed core M3 scope
Repository	Controlled GitHub history, protected main, PR workflow and meaningful reviews
CI	One functioning CI workflow that builds and runs selected automated checks
Automated tests	Minimum 15 meaningful functional/automated test cases overall: at least 5 unit/component, 5 black-box functional, 3 API/integration and 2 E2E/system; legitimate overlap is allowed.
Black-box evidence	Within the overall suite, use at least 2 appropriate black-box design techniques. Do not force every technique.
Integration/system evidence	At least 3 API/integration cases and 2 critical E2E/system cases within the overall minimum suite.
Static/security checks	At least one appropriate static/code-quality check and one security/dependency/secrets check where tool support permits
Performance	One small, justified performance/load test on a performance-sensitive operation
Staging/test deployment	One real OR credible simulated staging/test environment. Real cloud staging is optional and earns no automatic bonus marks.
Traceability	Evolving RTM links critical requirements → acceptance criteria → implementation → tests → results/evidence → defect/status.
Release review	One concise release-readiness summary with blockers, limitations and residual risks
PED	PED v3.0 updated as an evidence index, not a duplicated report

4. Required Deliverables

- Working CivicConnect release candidate / integrated increment.
- GitHub repository with authentic progressive history.
- Continue and evolve the existing PED v2.0 into PED v3.0. Do NOT create a new PED.
- Continue and evolve the existing Requirements Traceability Matrix (RTM); do NOT recreate it at the end of M3.
- Updated Risk Register.
- Updated Architecture Decision Records (ADRs) / decision log where significant decisions changed.
- Technical Debt Register.
- AI Usage Register.
- Test evidence and execution results.
- CI workflow and quality-gate evidence.
- Staging/test deployment link or assessor-accessible equivalent.
- Release Evidence Summary.
- Team presentation/demo and individual engineering defence.

5. Submission Requirements & Submission Procedure

M3 must be submitted as one controlled project baseline. Students must not create disconnected duplicate copies of evidence simply for submission. The PED v3.0 is the primary milestone document and evidence index; it should point the assessor to the authentic project evidence in the controlled repository and related project artefacts.

5.1 What Must Be Submitted

Submission Item	How It Is Submitted / Evidenced
PED v3.0	Submit the continued PED v2.0 as PED v3.0 in PDF format.

	This is the primary M3 milestone document and evidence index; do not create a separate M3 report.
GitHub repository URL	Submit the URL of the team's existing controlled repository. The repository must contain the authentic progressive implementation history, branches/Pull Requests, tests, CI configuration and relevant engineering artefacts.
M3 release candidate identifier	State the exact commit hash, tag or release/version that represents the M3 submission baseline. The assessor marks this identified version.
RTM and project registers	Continue the existing RTM, Risk Register, ADR/Decision records, Technical Debt Register, Defect Register and AI Usage Register. Keep them in their controlled project location and reference/link them clearly from PED v3.0. Do not duplicate them merely for submission.
Test, CI and quality evidence	Keep authentic evidence in the repository, CI platform, test reports or controlled artefact location. PED v3.0 should identify where the assessor can find the relevant evidence.
Staging/test environment	Provide the staging URL where externally accessible. If the team uses an assessor-accessible simulated/local staging arrangement, state clearly how it will be demonstrated during the presentation.
Team demonstration and individual defence	Completed live during the scheduled M3 presentation. The demonstration must use the identified M3 release candidate. No separate written presentation report is required unless specifically requested by the lecturer.

5.2 Moodle / Formal Submission

Unless the lecturer specifies a different Moodle configuration, the formal M3 submission should contain only the items needed to establish the assessment baseline:

- PED v3.0 — PDF.
- GitHub repository URL.
- M3 release candidate tag/commit/version identifier.
- Staging/test access information where applicable.

Source code does not need to be uploaded again as a ZIP when GitHub is the controlled repository. The assessed implementation is the exact repository commit/tag/version identified at submission.

5.3 Required Submission Information Block

Place the following information clearly near the beginning or end of PED v3.0 so that the assessor can locate the submitted baseline quickly:

- Team number/name and all team members.
- GitHub repository URL.
- M3 release candidate tag/commit/version.
- PED version: v3.0.
- RTM location/link.
- CI/test evidence location/link.
- Staging/test environment URL or demonstration instructions.
- Date submitted.

5.4 Submission Baseline Rule

The version assessed for M3 is the identified repository commit/tag/version. Changes made after the submission deadline are not part of the submitted milestone unless explicitly authorised. PED v3.0, the RTM and referenced evidence must correspond to that submitted release candidate.

Evidence should remain authentic and traceable. Do not replace repository/CI evidence with large collections of screenshots. Screenshots may support a specific claim where useful, but they should not become a second, disconnected evidence system.

6. PED v3.0 — Keep It as an Evidence Index

PED v3.0 is a continuation of the same Project Engineering Document used in earlier milestones. Do NOT start a new document. Preserve the project history, update existing sections as implementation and verification evidence becomes available, and record significant changes rather than silently replacing earlier decisions. The PED remains an evidence index: explain the engineering claim and point to authentic repository, CI, test and register evidence instead of duplicating screenshots.

PED v3.0 section	What to include
1. M3 overview	Continue PED v2.0 as PED v3.0; identify implemented scope, release candidate/build and current status. Preserve earlier project history.
2. M2 baseline conformance	What was implemented as planned; important deviations
3. Controlled changes	Significant changes, reason, impact and ADR/change reference
4. Construction/configuration	Repository strategy, configuration/secrets approach, important implementation notes
5. CI & automated verification	Pipeline summary, automated checks, gate rules, evidence links
6. Quality strategy applied	Which risks received stronger verification and why
7. Black-box/integration/system validation	Selected techniques, critical scenarios, results and evidence
8. Performance/reliability	Small workload test, results, limitations and relevant failure evidence
9. Defects & quality interpretation	Important defects/findings, status and implications
10. Release readiness	Gate summary, blockers, accepted limitations and residual risk
Appendices/links	Continue the existing RTM, Risk Register, ADRs, Technical Debt and AI Usage Register; add evidence links rather than recreating artefacts.

7. Assessment Area A — Controlled Implementation & M2 Baseline Conformance

What we are assessing
Can the team show that the implementation follows the approved design direction, or that important deviations were deliberately controlled rather than happening silently?

- Identify the release candidate/build being assessed.
- Show the main implemented components and how they relate to the M2 architecture/design.
- Demonstrate important persistence and API/integration decisions in the running system.
- Identify significant deviations from M2. Minor coding detail changes do not require an ADR.
- For a significant change, record: original decision → implementation evidence/problem → revised decision → consequence.
- Do not hide a justified change simply to appear consistent with M2.

Student-capacity evidence
A short baseline-conformance table plus 1–3 important ADR/change records is normally sufficient. Do not create an ADR for every small coding choice.

8. Assessment Area B — Collaborative Construction & Configuration Control

- Use meaningful feature/fix branches and Pull Requests.
- Protect main; no substantive direct development on main unless explicitly justified by an exceptional situation.
- Each substantive PR requires at least TWO approvals from team members other than the author, in line with the project brief.
- Reviews must contain evidence of actual checking; approval-only rubber stamping earns little/no review credit.
- Link issues/tasks to PRs where practical.
- Keep secrets out of the repository. Use environment variables, secret stores or equivalent configuration.
- Keep a progressive history; do not upload a completed system as one final commit.
- AI-assisted contributions must be understood, verified and recorded in the AI Usage Register.

Manageability
You are not marked on having hundreds of commits. We are looking for a believable engineering history and evidence that the team controlled change.

9. Assessment Area C — Automated Verification & CI Quality Gates

Meaningful Test Rule

A test counts toward the M3 minimum only when it has a traceable engineering basis (requirement, acceptance criterion, business rule, API contract, risk or defect), a clear purpose, deliberately selected test data, a defined expected result, a meaningful assertion/check, execution evidence and an interpretation of what the result demonstrates. Tests that merely duplicate another test, assert trivial behaviour, or exist only to increase the count do not count toward the minimum.

Overall M3 verification baseline	Minimum
Unit/component	5 meaningful cases
Black-box functional	5 meaningful cases
API/integration	3 meaningful cases
E2E/system	2 meaningful cases
Negative/failure/unauthorised behaviour	At least 3 cases across the overall suite
High-priority requirement/risk coverage	At least 5 cases across the overall suite
Black-box design techniques	At least 2 appropriate techniques across the suite
Performance/load	1 separate manageable scenario

Build a focused verification suite that protects important behaviour. M3 requires a minimum evidence baseline, but marks are awarded for engineering value, traceability, design, execution and interpretation — not for exceeding the minimum test count.

Required evidence	Manageable interpretation
Unit/component tests	At least 5 meaningful cases addressing important business rules, validation, calculations, permissions or component behaviour.
Regression tests	Retain meaningful corrected-defect and critical-behaviour tests within the overall suite; regression value matters more than a separate count.
API/integration automation	At least 3 important API/integration cases where practical, including failure/negative behaviour where relevant.
Static analysis	Run an appropriate code-quality/static-analysis tool or platform check.
Security/dependency/secrets check	Use a feasible automated check supported by the chosen ecosystem/GitHub.
CI workflow	Build + selected automated tests/checks run on PR/push as appropriate.
Quality gate	At least one meaningful failure must block progression, e.g. build/test failure or critical configured check.

Coverage rule
Coverage percentage may be reported, but M3 does not prescribe an arbitrary percentage. Students must explain what their coverage evidence does and does not demonstrate.

10. Assessment Area D — Black-Box, Integration & System Validation

Selection rule
Use techniques because the behaviour requires them. Do not manufacture artificial examples merely to tick every technique.

Technique/evidence	When it is useful
Equivalence Partitioning	Input groups expected to behave similarly
Boundary Value Analysis	Limits, ranges, dates, lengths, quantities
Decision Table	Several conditions interact to determine an outcome
State Transition	Expected behaviour depends on current/history state
Negative testing	Invalid, unauthorised, malformed or failure conditions
API/integration testing	Components/services must cooperate correctly
E2E/system testing	A critical user journey must work across the integrated system
Acceptance validation	Evidence that agreed business behaviour is satisfied

Minimum meaningful verification baseline
Across the project, provide at least 15 meaningful functional/automated test cases: ≥5 unit/component, ≥5 black-box functional, ≥3 API/integration and ≥2 E2E/system. Legitimate overlap is allowed. Across the overall set, at least 3 cases must exercise negative/failure/unauthorised behaviour, at least 5 must address high-priority requirements/risks, and at least 2 appropriate black-box design techniques must be represented. Add one separate manageable performance/load scenario. The minimum establishes workload, not quality.

11. What a Test Evidence Record Should Show

Field	Example of what belongs there
Test basis	Requirement, acceptance criterion, business rule, API contract or risk
Why selected	Risk/behaviour this test is intended to expose
Technique/level	BVA, decision table, API integration, E2E etc.
Input/precondition	Controlled test data and starting state
Expected result	Derived before execution from authoritative project evidence
Actual result	Observed behaviour
Status	Pass / fail / blocked
Evidence	Automated report, CI run, log, screenshot where appropriate, test record
Traceability	Requirement/issue/defect reference
Interpretation	What this result demonstrates — and important limitation if relevant

12. Assessment Area E — Quality, Defect & Traceability Evidence

- Continue the existing RTM throughout M3. Extend implemented requirements from stakeholder need and requirement through acceptance criteria, implementation, test case(s), execution evidence and defect/status. The RTM must show how verification evidence demonstrates the associated acceptance criteria; it must not be reconstructed retrospectively only for submission.
- Link critical implemented requirements to their acceptance criteria, implementation/component, relevant test case(s), expected result, execution evidence and current defect/status. One requirement may legitimately require several tests, and one well-designed test may support more than one criterion where the relationship is explicit.
- Maintain a simple defect record: defect → severity/impact → status → fix → regression evidence where corrected.
- Show important static/security findings and how they were handled.
- Interpret quality metrics rather than merely displaying them.
- Update the Technical Debt Register with important deferred compromises or known maintainability concerns.

Evolving RTM — mandatory
The RTM is a living project-control artefact carried forward from earlier milestones. By M3 its trace should evolve to: Stakeholder Need → Requirement →

Acceptance Criteria → Implementation/Component → Test Case(s) → Expected Result → Execution Evidence → Defect/Status. A test only provides useful traceability when it demonstrates the associated acceptance criterion. Git/repository history should support that the RTM evolved during the work rather than being assembled only at submission.

13. Assessment Area F — Performance & Basic Release Readiness

M3 requires a small performance/load exercise, not enterprise benchmarking.

- Choose ONE performance-sensitive operation or journey, for example request submission, dashboard loading, search/filtering or another justified operation.
- State why performance matters for this operation.
- Define a small but meaningful workload that your team can execute with available tools.
- Record environment, data assumptions, workload and duration.
- Measure at least response time and success/error behaviour; add throughput/resource evidence if the chosen tool makes this practical.
- Compare the result with a justified target or baseline. If no formal target exists, state that limitation rather than inventing one.
- Explain what the test does NOT prove about real municipal production scale.

Important
The assessor is evaluating whether you can design and interpret performance evidence — not whether three students can simulate an entire municipality.

14. Staging / Test Deployment

- Demonstrate the release candidate in an environment/configuration separate from normal development. Teams with the capacity may use a real cloud-hosted staging environment; teams may instead use a credible simulated staging environment that demonstrates environment-specific configuration, controlled secrets, release identification and verification of the deployed candidate.
- Use controlled configuration and do not expose secrets.
- Demonstrate that the correct release candidate is deployed.
- Run at least the critical E2E/system journey against the deployed environment where feasible.
- Identify configuration/environment differences that could affect production behaviour.
- Prepare a simple rollback/recovery approach: what version/artefact would you restore and what data/configuration concern must be considered?

Environment flexibility — no infrastructure bonus
Real cloud staging is optional in M3. A credible simulated staging environment is fully acceptable. Using paid cloud services, multiple servers, containers or other advanced infrastructure does not automatically earn additional marks. Marks are awarded for engineering evidence, configuration control, understanding and interpretation. Real production deployment is not required for M3.

15. Assessment Area G — Release-Quality Review & Residual Risk

The team must produce a concise Release Evidence Summary. This is where the separate pieces of evidence become an engineering argument.

Evidence area	Result	Interpretation	Concern / action
Build & CI	Pass/Fail	What configured checks demonstrate	Block/fix if mandatory gate fails
Automated regression	Pass/Fail/partial	Critical behaviour protected?	Investigate gaps/failures
API/integration	Result	Are critical integrations behaving?	Identify unresolved dependency risk
System/E2E	Result	Does critical journey complete?	Resolve or document limitation
Static/security	Findings	Severity/relevance	Fix, mitigate or record

Performance	Measured result	Against target/baseline and environment	Retest/record limitation
Known defects	Open/closed	Business impact	Block or residual risk
Acceptance	Evidence	Agreed behaviour demonstrated?	Missing acceptance evidence

Required final classification
READY — required evidence passes and no known release-blocking issue remains. CONDITIONALLY READY — core evidence is credible but explicitly identified non-blocking limitations/conditions remain. NOT READY — one or more important gates, critical behaviours or release-blocking risks remain unresolved. The classification itself earns little; the evidence and reasoning earn the marks.

16. Residual Risk Statement

Testing does not prove that no defect exists. Each team must identify a small number of meaningful residual risks that remain after M3 verification.

Residual risk	Why it remains	Current mitigation	Next action / owner
Example: production-scale load not tested	Student staging environment cannot represent municipal scale	Small load test + architecture review	Re-test in representative environment before real release
Example: external service uncertainty	Third-party sandbox behaviour may differ	Timeout/negative integration tests	Monitor/revalidate production integration

Do not invent risks to fill a table
Record real remaining uncertainty: untested paths, environment differences, known low-severity defects, external dependencies, deferred debt or other justified limitations.

17. Individual Engineering Defence — 20 Marks

Every student is assessed individually. A team member must be able to explain the team's engineering work, not only the part they personally typed.

Defence area	What the student should be able to do	Marks
Requirement-to-evidence trace	Trace one requirement through design/implementation/test/evidence	4
Engineering decision	Explain one important decision, alternative and consequence	4
Quality evidence interpretation	Interpret a test/CI/static/performance result and its limitation	4
Failure/risk reasoning	Explain one realistic failure, control and remaining risk	4
Release readiness defence	Use evidence to explain the team's readiness classification	4

Individual mark rule
Being present at the presentation does not guarantee the individual marks. Marks depend on demonstrated understanding and the ability to defend authentic project evidence.

18. Suggested Presentation / Demonstration Flow

Stage	Suggested evidence	Guide
1. Release candidate	Identify build/version and implemented scope	~2 min
2. Baseline → implementation	Architecture/design conformance + significant change	~3 min

3. Controlled construction	GitHub/PR/configuration evidence	~3 min
4. CI & automated assurance	Run/report + gate evidence	~4 min
5. Black-box/integration/system	Selected high-value tests + results	~5 min
6. Staging + critical journey	Short live demonstration	~4 min
7. Performance/release evidence	Small load result + release summary + residual risk	~4 min
8. Individual defence	Assessor questions each student	Separate individual assessment

Do not spend the presentation clicking through every screen
The application demo is evidence, not the entire assessment. Prioritise engineering decisions, verification and interpretation.

19. Marking Rubric — Team Evidence (30 Marks)

Criterion	Marks	High-quality evidence	Partial evidence	Weak / missing evidence
A. Implementation & M2 baseline conformance	5	Working integrated scope; clear baseline link; significant deviations justified/controlled	Mostly working; some baseline/change evidence unclear	Little integration; silent/unexplained deviation
B. Collaborative construction & configuration	4	Authentic progressive Git/PR/review history; main protected; configuration/secrets controlled	Workflow exists but inconsistent or review evidence weak	Late dump/direct-main/rubber-stamp reviews/secrets risk
C. Automated verification & CI gates	6	≥15 meaningful cases with required mix/overlap; traceable to ACs/risks/defects; useful assertions; CI/gates work; results interpreted	Minimum mostly present but weak traceability, risk coverage, assertions or CI integration	Count-padding, duplicated/shallow tests, manual-only evidence or CI absent/unreliable
D. Black-box, integration & system validation	6	Appropriate techniques selected; critical API/integration/E2E evidence; expected results traceable	Useful tests but gaps, weak technique justification or limited integration evidence	Mostly happy-path/screenshots; poor expected-result basis
E. Quality, defect & traceability evidence	4	Existing RTM visibly evolved: requirement → AC → implementation → tests → evidence → defect/status; defects/findings/debt analysed	Artefacts updated but trace chain incomplete or appears largely retrospective	New/recreated RTM/PED, evidence dump, or little acceptance-criteria traceability
F. Performance & staging/release readiness	3	Manageable justified load test + real or simulated staging evidence + limitations interpreted; infrastructure choice justified	Some environment/performance evidence but weak workload/configuration/interpretation	No credible environment/performance evidence or complexity shown without engineering value
G. Release review & residual risk	2	Evidence-based classification; real blockers/limitations/residual risks visible	Classification given but reasoning shallow	Unsupported 'ready' claim or risks hidden

20. Marking Rubric — Individual Defence (20 Marks)

Criterion	4 — Strong	3 — Competent	2 — Developing	1–0 — Weak/No evidence
Requirement-to-evidence trace	Accurately traces requirement → implementation → test → evidence and explains link	Mostly accurate with minor prompting	Partial trace; weak links	Cannot trace authentic evidence
Engineering decision	Explains context,	Explains decision and basic	Describes decision but	Cannot justify / appears

	alternative, decision and consequence	rationale	limited trade-off reasoning	unfamiliar
Quality evidence interpretation	Explains what evidence proves and does not prove	Correct interpretation with minor gaps	Reads result but weak interpretation	Treats screenshot/green status as proof
Failure/risk reasoning	Explains failure, impact, control and residual risk	Identifies credible risk/control	Risk explanation generic	Cannot reason about failure
Release readiness defence	Uses several evidence sources to defend classification	Uses relevant evidence with minor gaps	Opinion stronger than evidence	Cannot support team's classification

21. What Will Not Earn Strong Marks

- A large number of screenshots with no explanation of the engineering claim they support.
- An application demo with no traceable test/quality evidence.
- Artificial use of every test technique simply because it appeared in the notes.
- Coverage percentage presented as proof of correctness.
- CI that exists but does not run meaningful checks.
- Repeatedly rerunning flaky tests until they pass.
- Performance numbers with no workload/environment context.
- Calling the system 'production ready' while important failed tests or high-impact defects are hidden.
- One team member answering every technical question while others cannot explain the project.
- AI-generated tests/code that team members cannot explain or verify.

22. Recommended Team Work Plan

Step	Team focus	Keep it manageable
1. Freeze M3 core scope	Agree which implemented requirements form the release candidate	Do not keep adding features
2. Reconcile with M2	Identify only significant deviations	Create ADR/change records only where needed
3. Stabilise integration	Make core journeys work before expanding tests	Fix integration before polishing
4. Select quality risks	Choose the most important behaviours/failures	Risk-based, not test-count based
5. Automate core regression	Unit/API/integration checks for important behaviour	Small reliable suite
6. Configure CI	Build + automated checks + meaningful gate	One working pipeline is enough
7. Design black-box/system evidence	Select applicable techniques and critical journey	Reuse scenarios where legitimate
8. Deploy staging/test	Deploy one controlled release candidate	No live production requirement
9. Run small performance test	One justified operation/workload	Interpret limitations
10. Update evidence artefacts	PED v3, RTM, defects, risks, ADR/debt/AI registers	Link rather than duplicate
11. Conduct release review	Classify readiness and residual risk	Be evidence-based
12. Prepare defence	Every student rehearses trace/decision/evidence/risk	No memorised script only

Expected Assessor Questions — Team & Individual Engineering Defence

The presentation is not only a demonstration of a working system. The assessor may question any team member to determine whether the engineering decisions, implementation, verification evidence and release-candidate status are genuinely understood. Questions may be adapted to the team's actual product and evidence; students should therefore prepare to reason from their own project rather than memorise model answers.

Important: These are representative question types, not a fixed question bank. The assessor may follow up an answer, ask a student to locate evidence in the PED/RTM/GitHub/CI output, or change the scenario to test engineering judgement.

1. Requirement-to-Evidence Trace

- Choose one important requirement. Trace it from the stakeholder need and acceptance criterion to the implemented functionality and the test evidence.
- Where in the RTM can I see that this requirement has actually been implemented and verified?
- Why is the evidence you are showing sufficient for this acceptance criterion?
- If this test passed but the acceptance criterion was still not satisfied, what could explain the difference?

2. Engineering Decision & M2 Baseline Conformance

- Show one significant architecture, design or technology decision from M2 that can now be seen in the implementation.
- Did the implementation remain consistent with the M2 baseline? If not, what changed and where was that change controlled?
- What trade-off did this decision create for maintainability, integration, testing, performance or deployment?
- If you were making this decision again with what you now know, would the reasoning remain valid? Explain using project evidence.

3. Collaborative Construction & Configuration Control

- Show a Pull Request that demonstrates meaningful peer review. What changed because of the review?
- Why is the commit/tag you identified the release candidate being assessed?
- How do you know the demonstrated product corresponds to the submitted repository baseline?
- What engineering risk would arise if team members developed directly on the protected main branch without controlled review?

4. Automated Verification & CI Quality Gates

- Show one CI run. What checks were executed, and what does a successful run actually allow you to conclude?
- Which important quality concern is not proven merely because the CI pipeline is green?
- Show a failed check or defect discovered during verification. What did the team do with that evidence?
- If an automated test becomes flaky, should the team simply rerun it until it passes? Explain the engineering consequence.

5. Black-Box, Integration & System Validation

- Show a black-box test and explain the test-design reasoning behind the chosen inputs.
- Show an integration or API test. Which boundary or interaction is being verified?
- Demonstrate one negative, failure or unauthorised behaviour. Why is that behaviour important to verify?
- What is the difference between showing that a screen works and demonstrating that the associated requirement has been validated?

6. Quality Evidence, Defects & Traceability

- Which defect currently creates the greatest concern for this release candidate, and why?
- Show how a defect discovered during testing is connected to the relevant requirement, test or risk.
- Which requirement has the strongest verification evidence? Which has the weakest? Explain.
- If you had one additional day before release, where would you focus verification effort and why?

7. Performance, Staging & Release Readiness

- What workload or performance-sensitive behaviour did you evaluate, and why was it relevant to this system?
- What does your performance evidence allow you to conclude, and what does it not allow you to conclude?
- How does your staging or simulated-staging environment differ from development?
- What would have to be true before you would recommend this release candidate for actual production use?

8. Failure, Risk & Residual Risk Reasoning

- Identify one important residual risk that remains even though the related tests passed.

- Suppose one of your external dependencies becomes unavailable. What part of the system would be affected and what evidence would you want?
- If a critical test fails immediately before release, what factors would determine whether release should proceed?
- What technical debt are you knowingly carrying into the next milestone, and what future consequence could it create?

9. AI-Assisted Engineering & Accountability

- Identify one meaningful use of AI in the project. How did you verify the output before accepting it?
- Show an example where AI-generated output was changed, rejected or corrected by the team.
- If AI produced code that passed the tests, why would human engineering judgement still be required?
- Who is accountable for an AI-assisted implementation or test that later proves incorrect?

10. Individual Contribution & Understanding

- Which part of the release candidate can you personally explain in detail, and what engineering decisions did you contribute to?
- Show evidence of your contribution in the controlled project history.
- Explain one decision made by another team member that affects your own work.
- If your teammate were unavailable, could you explain how this part of the system is verified and changed safely?

How Students Should Prepare

- Do not memorise definitions. Prepare to navigate your own PED v3.0, RTM, GitHub repository, Pull Requests, CI runs, tests, defects and release-candidate evidence.
- Each student should be able to trace at least one important requirement end-to-end and explain the evidence rather than only pointing to it.
- Be prepared for follow-up questions such as “Why?”, “What evidence supports that?”, “What changed?”, “What is the consequence?” and “What would you do if this failed?”
- A correct technical statement without project evidence may demonstrate knowledge, but the defence is intended to establish applied engineering understanding.

Assessor Questioning Principle

The assessor does not need to ask every student the same questions. Questions should sample the five individual assessment dimensions: requirement-to-evidence trace, engineering decision, quality-evidence interpretation, failure/risk reasoning, and release-readiness defence. Follow-up questions may be used where an answer is rehearsed, unsupported, unclear or inconsistent with the project evidence.

23. Final Submission Checklist

☐ Release candidate/build is identifiable and demonstrable.

☐ Core agreed M3 scope is integrated.

☐ M2 baseline deviations are recorded where significant.

☐ GitHub main is controlled and substantive PRs show required review.

☐ No secrets are knowingly committed.

☐ CI build/check workflow runs.

☐ At least 15 meaningful functional/automated test cases are evidenced, with appropriate overlap across required categories.

☐ Appropriate black-box techniques are demonstrated.

☐ At least two important API/integration scenarios are evidenced.

☐ At least one critical E2E/system journey is evidenced.

☐ Static/code-quality and feasible security checks are evidenced.

☐ One manageable performance/load exercise is documented and interpreted.

☐ A real or simulated staging/test environment is demonstrable; infrastructure complexity has not been treated as a source of bonus marks.

☐ The existing RTM has evolved continuously and links critical requirements → acceptance criteria → implementation → tests → execution evidence → defect/status.

☐ Defect, Risk, Technical Debt, ADR and AI registers are updated as applicable.

☐ Release Evidence Summary is complete.

☐ Residual risks are stated honestly.

☐ The existing PED v2.0 has been continued as PED v3.0, preserving project history and pointing to authentic evidence rather than duplicating it.

☐ PED v3.0 PDF is ready as the primary submitted milestone document/evidence index.

☐ GitHub repository URL is provided and accessible to the assessor.

☐ Exact M3 release candidate tag/commit/version is stated and matches the demonstrated product.

☐ RTM and existing project registers remain in their controlled locations and are clearly referenced from PED v3.0.

☐ CI/test evidence and staging/test access information can be located from the submitted PED/repository.

☐ No unnecessary duplicate ZIP/source-code submission has been created where GitHub is the controlled repository.

☐ Every team member is ready for individual engineering defence.

24. Final Evidence Principle

M3 defence pattern
Stakeholder need → Requirement → Acceptance Criterion → Engineering decision → Implementation → Test → Execution evidence → Defect/Risk → Release readiness.

M3 defence pattern
Requirement → engineering decision → implementation → verification → evidence → defect/risk → release readiness.